รายงานผลการทดลอง Lab 3

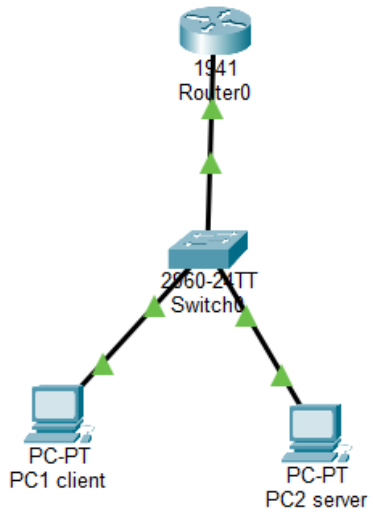
สมาชิก

673380056-7 รพีภัทร แก้วคูณ (ไผ่)
673380068-0 สุชาดา ศรีจักรโคตร (เค้ก)
673380286-0 พิไลพร คำเวียง (เอวา)

Addressing Scheme

Device	Interface	IP	Gateway
R1	G0/0.10	192.168.10.1/24	—
R1	G0/0.20	192.168.20.1/24	—
PC1-Client	NIC	192.168.10.10	192.168.10.1
PC2-Server	NIC	192.168.20.20	192.168.20.1

Topology

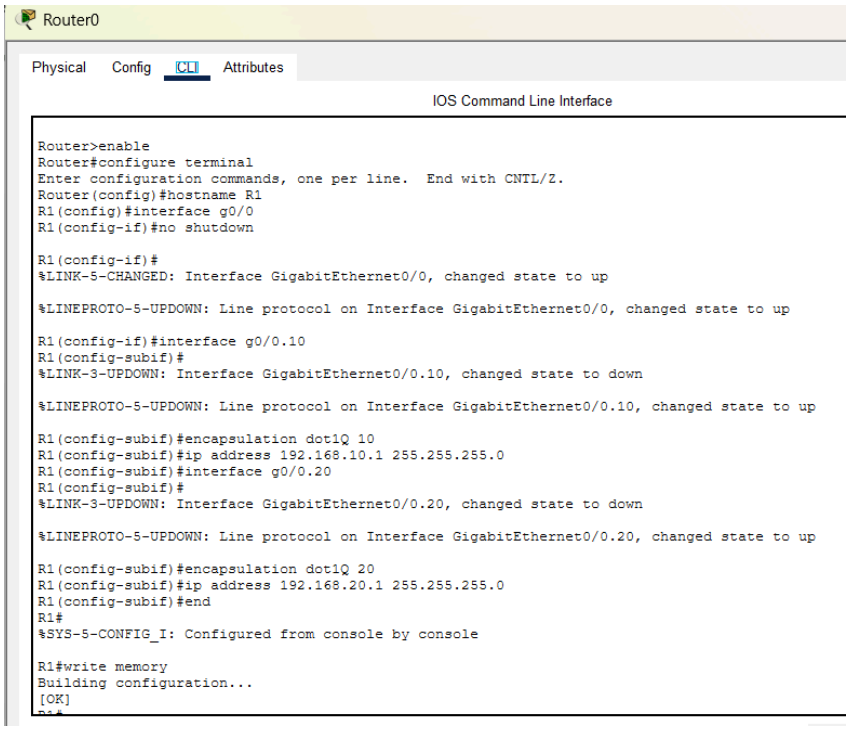


Part 1 — Configure Network Devices

Step 1 — Router Configuration

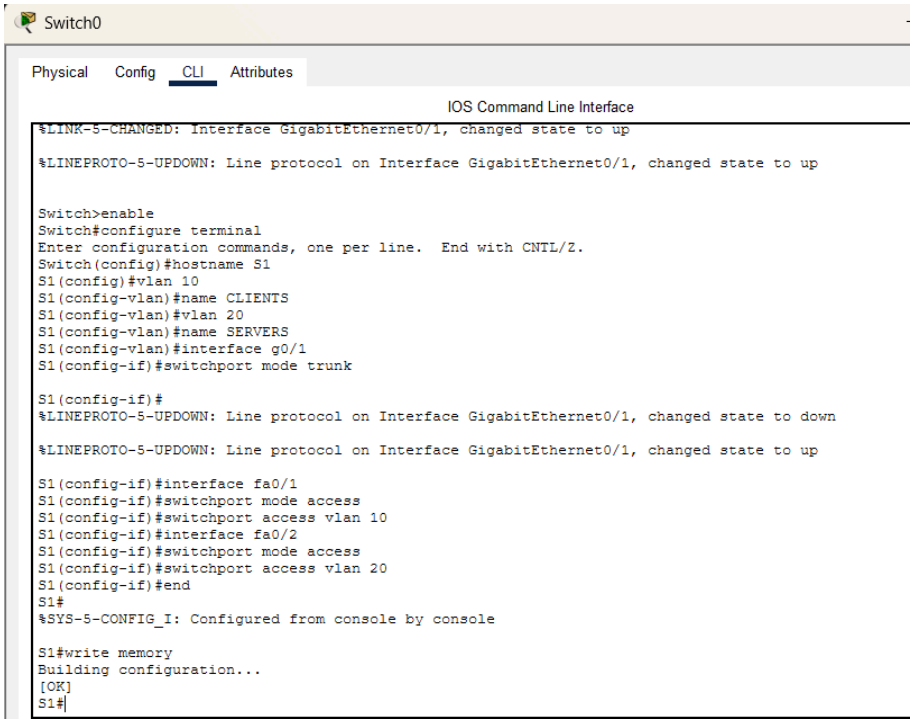
ทำการกำหนดค่าา

```
enable
configure terminal
hostname R1

interface g0/0/0
no shutdown

interface g0/0/0.10
encapsulation dot1Q 10
ip address 192.168.10.1 255.255.255.0

interface g0/0/0.20
encapsulation dot1Q 20
ip address 192.168.20.1 255.255.255.0

end
write memory
```



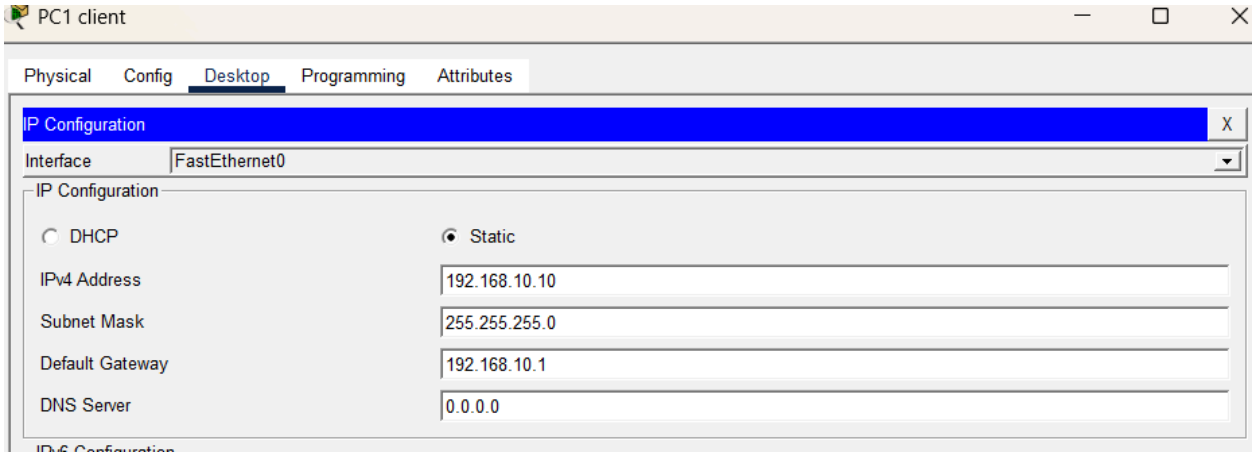
Step 2 — Switch Configuration

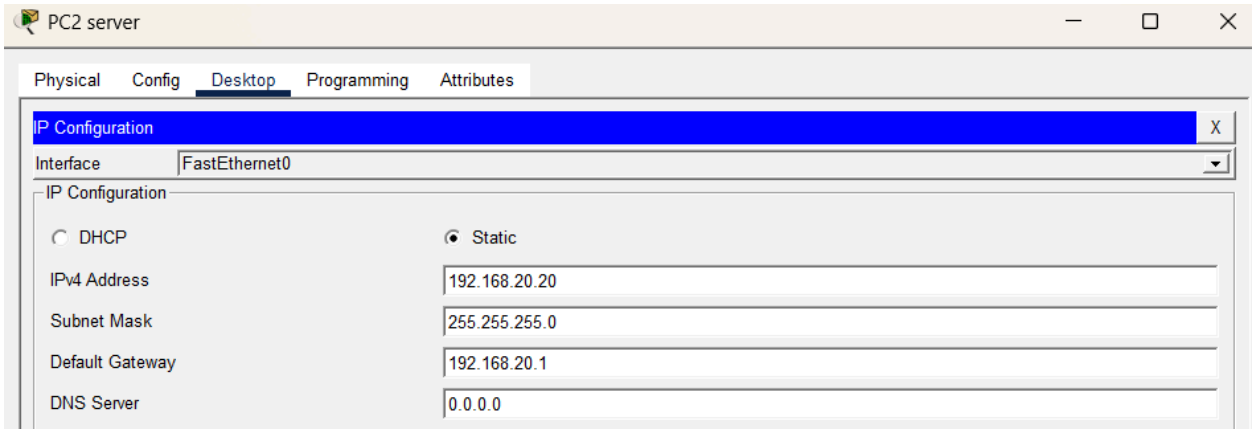
```
enable
configure terminal
hostname S1

vlan 10
name CLIENTS

vlan 20
name SERVERS

interface g1/0/11
switchport mode trunk

interface g1/0/23
switchport mode access
switchport access vlan 10

interface g1/0/24
switchport mode access
switchport access vlan 20

end
write memory
```



Step 3 — Configure Hosts

Set static IP addresses per table.





Part 2 — Verify Connectivity

From PC1:

```
ping 192.168.20.20  พบว่า ping สำเร็จ มีการตอบกลับ 100%
```

PC1 client

```
C:\>ping 192.168.20.20

Pinging 192.168.20.20 with 32 bytes of data:

Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time=1ms TTL=127

Ping statistics for 192.168.20.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms
```

PC1 client

```
C:\>arp -a
  Internet Address      Physical Address      Type
  192.168.10.1          0001.9703.6e01        dynamic

C:\>
```

จากคำสั่ง arp -a พบว่าเครื่อง PC1 มี ARP entry ของ IP 192.168.10.1 ซึ่งเป็น Default Gateway โดยแสดง MAC Address แบบ dynamic แสดงว่าเครื่องสามารถค้นหา MAC Address ของ Router ได้สำเร็จ และการสื่อสารภายใน VLAN 10 ทำงานปกติ

PC2 server

Physical Config Desktop Programming Attributes

Command Prompt

```
Cisco Packet Tracer PC Command Line 1.0
C:\>arp -a
  Internet Address      Physical Address      Type
  192.168.20.1          0001.9703.6e01        dynamic
```

จากคำสั่ง arp -a พบว่าเครื่อง PC1 มี ARP entry ของ IP 192.168.20.1 ซึ่งเป็น Default Gateway โดยแสดง MAC Address แบบ dynamic แสดงว่าเครื่องสามารถค้นหา MAC Address ของ Router ได้สำเร็จ และการสื่อสารภายใน VLAN 10 ทำงานปกติ

Router checks:

show ip route

```
R1>show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/0.10
L       192.168.10.1/32 is directly connected, GigabitEthernet0/0.10
     192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/0.20
L       192.168.20.1/32 is directly connected, GigabitEthernet0/0.20
```

show ip interface brief

```
R1>show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  up                    up
GigabitEthernet0/0.10  192.168.10.1    YES manual up                    up
GigabitEthernet0/0.20  192.168.20.1    YES manual up                    up
GigabitEthernet0/1     unassigned      YES unset  down                  down
Vlan1                  unassigned      YES unset  up                    down
R1>
```

พบว่า

- GigabitEthernet0/0 อยู่ในสถานะ up/up
- GigabitEthernet0/0.10 และ 0/0.20 มี IP Address ถูกต้อง และสถานะ up/up

แสดงว่า Subinterface สำหรับ VLAN 10 และ VLAN 20 ทำงานปกติ และ Router พร้อมรับส่งข้อมูลระหว่าง VLAN ได้อย่างถูกต้อง

Switch checks:

show interfaces trunk

```
S1>show interfaces trunk
Port        Mode         Encapsulation  Status        Native vlan
Gig0/1      on           802.1q         trunking      1

Port        Vlans allowed on trunk
Gig0/1      1-1005

Port        Vlans allowed and active in management domain
Gig0/1      1,10,20

Port        Vlans in spanning tree forwarding state and not pruned
Gig0/1      1,10,20
```

show mac address-table

```
S1>show mac address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----

   1    0001.9703.6e01    DYNAMIC     Gig0/1
  10    0001.9703.6e01    DYNAMIC     Gig0/1
  20    0001.9703.6e01    DYNAMIC     Gig0/1
S1>
```

พบว่า MAC Address 0001.9703.6e01 ถูกเรียนรู้แบบ Dynamic ผ่านพอร์ต Gig0/1 และปรากฏใน VLAN 10 และ VLAN 20

แสดงว่า trunk port สามารถส่งข้อมูลหลาย VLAN ได้ถูกต้อง และ Switch สามารถเรียนรู้ MAC Address ของ Router ในแต่ละ VLAN ทำให้การสื่อสารระหว่าง VLAN ทำงานปกติ

Part 3 — Deploy MIME Server

On Server (PC2)

```
cd automation
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```

```
C:\Users\korpa\OneDrive\เดสก์ ท็ อป\Week_01\week01-mime-typing>python server/main_enhanced.py --host 0.0.0.0 --port 65432
--verbose
```

```
INFO: Starting server with host=0.0.0.0, port=65432, storage=storage
INFO: Server listening on 0.0.0.0:65432
```

On Client (PC1)

```
cd automation
python client/main_enhanced.py assets/sample.png --host 192.168.20.20 --port 65432
```

```
127.0.0.1 --port 65432 --verbose
```

```
INFO: Starting client with 1 files to 127.0.0.1:65432
INFO: Connected to 127.0.0.1:65432
INFO: Sent ../assets/sample.png (108 bytes) as image/png
INFO: Connection closed. Sent: 1, Failed: 0
INFO: All files sent successfully
```

ในฝั่ง clients :

```
INFO: Receiving image/png (108 bytes)
INFO: Saved to storage\received_a0ed.png
```

Confirm file saved.

Part 4 — Wireshark Capture Walkthrough on PC1 and PC1

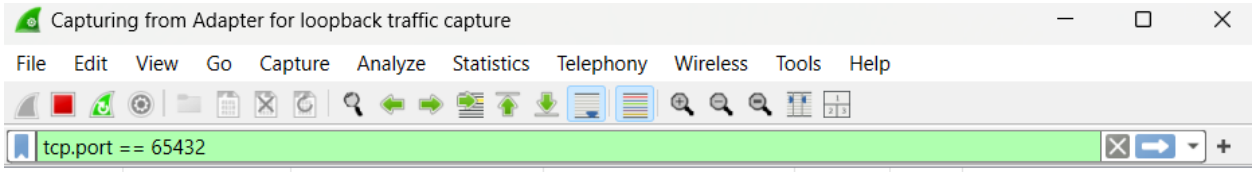
Step 1 — Start Capture

Open Wireshark.

Select interface connected to network.

Apply capture filter:

```
tcp port 65432
```



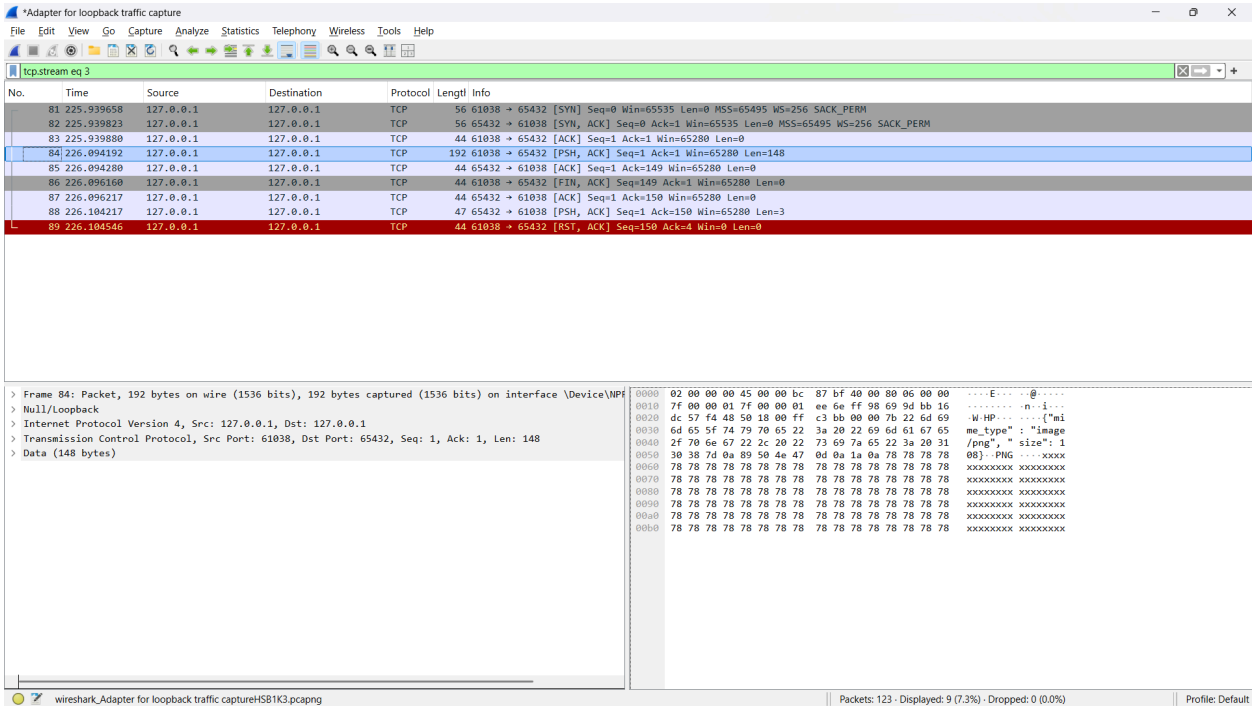
Step 2 — Initiate Transfer

Send file again.

```
127.0.0.1 --port 65432 --verbose
```

```
INFO: Starting client with 1 files to 127.0.0.1:65432
INFO: Connected to 127.0.0.1:65432
INFO: Sent ../assets/sample.png (108 bytes) as image/png
INFO: Connection closed. Sent: 1, Failed: 0
INFO: All files sent successfully
```

Stop capture after completion.



จากการ Capture แพ็กเก็ตด้วย Wireshark โดยกำหนด Display Filter เป็น `tcp.port == 65432` พบว่า Client (Source Port 65524) ทำการเชื่อมต่อไปยัง Server (Destination Port 65432) ผ่าน IP Address 127.0.0.1 ซึ่งเป็น Loopback Interface

TCP Three-Way Handshake ประกอบด้วย

- แพ็กเก็ต **SYN** จาก Client ไปยัง Server (65524 → 65432)
- แพ็กเก็ต **SYN, ACK** จาก Server ตอบกลับ
- แพ็กเก็ต **ACK** จาก Client

จากลำดับข้างต้นแสดงให้เห็นว่าการสร้างการเชื่อมต่อ TCP สำเร็จและทั้งสองฝั่งพร้อมสำหรับการรับและส่งข้อมูล

หลังจากการเชื่อมต่อถูกสร้างแล้ว พบ Packet ประเภท **PSH, ACK** ซึ่งบ่งบอกถึงการส่งข้อมูล (Payload) จาก Client ไปยัง Server โดยภายใน Payload ประกอบด้วย JSON Header ที่ระบุ MIME type และขนาดไฟล์ ตามด้วยข้อมูลไบนารีของไฟล์จริง ทั้งนี้ TCP จะมีPacket **ACK** ตอบกลับเพื่อยืนยันการรับข้อมูล

เมื่อการส่งข้อมูลเสร็จสิ้น พบแพ็กเก็ต **FIN, ACK** เพื่อเริ่มต้นการปิดการเชื่อมต่อ และมีแพ็กเก็ต **ACK** ตอบกลับ จากนั้นจะเห็นแพ็กเก็ต **RST, ACK** ซึ่งเป็นการปิดการเชื่อมต่อ

เนื่องจากการทดสอบครั้งนี้ใช้ IP 127.0.0.1 (Loopback Interface) การสื่อสารทั้งหมดเกิดขึ้นภายในเครื่องเดียวกัน จึงไม่พบการทำงานของ ARP ในการ Capture

---

Step 3 — Analyze Packets

A — ARP Exchange

Look for:

- Who has 192.168.20.20?

Confirm MAC resolution.

ในการ Capture ครั้งนี้ไม่พบ ARP Exchange เนื่องจากการสื่อสารเกิดขึ้นผ่าน Loopback Interface (127.0.0.1) ซึ่งไม่ต้องใช้ MAC Address Resolution และไม่ผ่าน Data Link Layer แบบ Ethernet

---

B — TCP Three-Way Handshake

Identify:

1 SYN 2 SYN-ACK 3 ACK

Check sequence numbers.

No.	Time	Source	Destination	Protocol	Lengtl	Info
81	225.939658	127.0.0.1	127.0.0.1	TCP	56	61038 → 65432 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
82	225.939823	127.0.0.1	127.0.0.1	TCP	56	65432 → 61038 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
83	225.939880	127.0.0.1	127.0.0.1	TCP	44	61038 → 65432 [ACK] Seq=1 Ack=1 Win=65280 Len=0
84	226.094192	127.0.0.1	127.0.0.1	TCP	192	61038 → 65432 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=148
85	226.094280	127.0.0.1	127.0.0.1	TCP	44	65432 → 61038 [ACK] Seq=1 Ack=149 Win=65280 Len=0
86	226.096160	127.0.0.1	127.0.0.1	TCP	44	61038 → 65432 [FIN, ACK] Seq=149 Ack=1 Win=65280 Len=0
87	226.096217	127.0.0.1	127.0.0.1	TCP	44	65432 → 61038 [ACK] Seq=1 Ack=150 Win=65280 Len=0
88	226.104217	127.0.0.1	127.0.0.1	TCP	47	65432 → 61038 [PSH, ACK] Seq=1 Ack=150 Win=65280 Len=3
89	226.104546	127.0.0.1	127.0.0.1	TCP	44	61038 → 65432 [RST, ACK] Seq=150 Ack=4 Win=0 Len=0

จากการวิเคราะห์แพ็กเก็ตใน Wireshark พบว่าการสื่อสารระหว่าง Client และ Server บนพอร์ต 65432 เป็นไปตามมาตรฐานของโปรโตคอล TCP อย่างครบถ้วน โดยในช่วงเริ่มต้นจะเห็นการสร้างการเชื่อมต่อก่อนมีการส่งข้อมูลจริง

ขั้นตอนแรกคือ Client ส่งคำขอเชื่อมต่อไปยัง Server ซึ่งปรากฏในรูปแบบของแพ็กเก็ต SYN จากนั้น Server ตอบกลับด้วยแพ็กเก็ตที่มีสถานะ SYN และ ACK พร้อมกัน เพื่อยืนยันว่าพร้อมรับการเชื่อมต่อ สุดท้าย Client ส่ง ACK กลับไปอีกครั้ง ทำให้การเชื่อมต่อเข้าสู่สถานะพร้อมใช้งานกระบวนการนี้สะท้อนถึงการทำงานของ TCP 3-Way Handshake อย่างสมบูรณ์ โดยค่า Sequence Number และ Acknowledgment Number มีการเพิ่มค่าอย่างสัมพันธ์กันตามลำดับของ segment ที่ถูกส่งและยืนยัน

หลังจากการเชื่อมต่อถูกสร้างขึ้นแล้ว จึงเริ่มมีการส่งข้อมูลจริง โดยปรากฏแพ็กเก็ตที่มี Flag เป็น PSH, ACK ซึ่งบ่งชี้ว่ามีการส่ง Payload ไปยังอีกฝั่ง และมีการตอบรับด้วย ACK เพื่อยืนยันการรับข้อมูลตามกลไกความน่าเชื่อถือของ TCP

เมื่อการรับ–ส่งข้อมูลเสร็จสิ้น จะเข้าสู่กระบวนการปิดการเชื่อมต่อ โดยมีแพ็กเก็ต FIN, ACK เพื่อแจ้งยุติการสื่อสาร และตามด้วย ACK จากอีกฝั่ง ก่อนที่การเชื่อมต่อจะถูกยุติอย่างสมบูรณ์ด้วย RST, ACK แสดงถึงการสิ้นสุด session การสื่อสารระหว่างทั้งสองฝ่าย

C — Data Transfer

Follow stream:

Right click → Follow → TCP Stream.

You should see JSON header similar to:

```
{"mime_type":"image/png","size":12345}
```

Followed by binary.

การวิเคราะห์ TCP Stream พบ JSON Header ที่ระบุ MIME type และขนาดไฟล์ ก่อนข้อมูลไบนารี ซึ่งเป็นไปตามการออกแบบของ Application Protocol

---

D — VLAN Tags

If capturing on trunk:

Look for:

```
802.1Q Virtual LAN
```

Note VLAN ID.

ไม่พบ 802.1Q VLAN Tag ในการ Capture ครั้งนี้ เนื่องจากการทดสอบไม่ได้เกิดขึ้นบน trunk interface หรือสภาพแวดล้อมที่มี VLAN segmentation

---

Packet Dissection Guide

Layer	Field	What to Observe
Ethernet	Src/Dst MAC	Frame direction
802.1Q	VLAN ID	Segmentation
IP	TTL	Routing behavior
TCP	Flags	Reliability
Payload	JSON	MIME metadata

Part 5 — Troubleshooting Labs

Scenario 1 — Broken Routing

Shutdown subinterface.

Observe:

- Ping failure
- TCP timeout

จากการสังเกต เมื่อ shutdown subinterface บน Router ทำให้ไม่สามารถติดต่อสื่อสารข้าม VLAN ถึงอุปกรณ์ในระดับ Layer 2 เช่น Switch ยังคงทำงานตามปกติ แต่เมื่อ Layer 3 ไม่ทำงาน Routing Decision จึงไม่เกิดขึ้น แพ็กเก็ตไม่ถูกส่งต่อไปยังปลายทาง ทำให้กระบวนการ TCP Handshake ไม่สมบูรณ์ (timeout) ฝั่งผู้ส่งจะเห็นเพียงการส่ง SYN ออกไป แต่ไม่ได้รับ SYN-ACK กลับมา

Scenario 2 — Firewall Block

Block port.

Observe:

- SYN retransmissions

จากการสังเกต เมื่อกำหนด Firewall ให้บล็อกพอร์ตที่ใช้ในการสื่อสาร แม้ว่า Layer 3 จะทำงานได้ตามปกติ แต่การกรองทราฟฟิกในระดับ Layer 4 ส่งผลให้แพ็กเก็ต TCP ถูกปฏิเสธโดยไม่มีการตอบกลับ ทำให้ฝั่งผู้ส่งเข้าใจว่าแพ็กเก็ตสูญหายและดำเนินการส่งซ้ำ

Scenario 3 — Wrong Gateway

Observe:

- ARP incomplete

จากการสังเกต การตั้งค่า Default Gateway ไม่ถูกจะส่งผลกระทบตั้งแต่ขั้นตอนเริ่มต้นของการสื่อสาร ก่อนที่ TCP จะเริ่มต้นทำงาน เครื่องต้นทางจำเป็นต้องทราบ MAC Address ของ Gateway ผ่านกระบวนการ ARP หากไม่มีอุปกรณ์ใดตอบกลับ ARP Request การ

สร้าง Ethernet Frame จะไม่สามารถเกิดขึ้นได้ ส่งผลให้ไม่สามารถส่งแพ็กเก็ตออกจากเครือข่ายย่อยได้เลย

Scenario 4 — Packet Loss Simulation

Drop packets.

Observe:

- TCP retransmissions
- Window adjustment

จากการสังเกต การจำลองการสูญหายของแพ็กเก็ตระหว่างทาง เมื่อมีการสูญหายของข้อมูล ฝั่งผู้รับจะตรวจพบลำดับหมายเลข (Sequence Number) ที่ขาดหายและส่ง Duplicate ACK กลับไปยังผู้ส่ง ผู้ส่งจึงทำการ Retransmit ข้อมูล พร้อมทั้งปรับขนาดหน้าต่างรับส่งข้อมูลและควบคุมอัตราการส่งตามหลัก Congestion Control และ Flow Control กลไกดังกล่าวทำให้แม้เกิดความผิดพลาดในเครือข่าย การสื่อสารยังคงสามารถดำเนินต่อไปจนเสร็จ

Lab Report Requirements

Include:

- Screenshots of configurations
- Ping results
- Wireshark capture screenshots
- JSON header evidence
- Encapsulation explanation
- Troubleshooting notes

Assessment Questions

1. Why does TCP retransmit lost segments?
   Ans. TCP มีการส่งข้อมูลซ้ำ (retransmit) เพราะเป็นโปรโตคอลแบบ เชื่อถือได้ (Reliable) และมีการเชื่อมต่อ (Connection-Oriented)
2. How does VLAN tagging affect broadcast domains?
   Ans. VLAN Tagging (802.1Q) ทำให้ Switch หนึ่งตัวถูกแบ่งออกเป็นหลาย Broadcast Domain

3. What role does MIME play compared to TCP headers?
   Ans. TCP Header ทำหน้าที่ควบคุม การส่งข้อมูล
   MIME ทำหน้าที่กำหนด ความหมายของข้อมูล
4. Where does routing decision occur?
   Ans. เพราะการตัดสินใจเส้นทาง (Routing Decision) เกิดที่ Router (Layer 3)
5. Why does ARP precede TCP communication?
   Ans. ก่อนเริ่ม TCP ต้องรู้ MAC Address ของปลายทางก่อน

Reflection Exercise

Explain:

How does application meaning survive multiple encapsulation layers?
= Application meaning สามารถคงอยู่ได้แม้ผ่านกระบวนการ encapsulation หลายชั้นเนื่องจากแต่ละ layer เช่น TCP/IP จะเพิ่มเพียง header ของตนเองเข้าไป โดยไม่แก้ไขข้อมูลของชั้นบน ข้อมูลจาก Application Layer จะถูกเก็บไว้ในรูปแบบ payload ของแต่ละชั้นที่ห่อหุ้มลงไปเรื่อย ๆ จนถึงการส่งผ่านสื่อกลางเมื่อข้อมูลถึงปลายทาง จะเกิดกระบวนการ decapsulation ออกตามลำดับย้อนกลับ ทำให้ข้อมูลต้นฉบับจาก Application Layer ถูกส่งถึงโปรแกรมปลายทางอย่างครบถ้วน ดังนั้น ความหมายของข้อมูลจึงไม่สูญหาย เพราะแต่ละ layer แยกหน้าที่ชัดเจนและไม่ตีความหรือดัดแปลงเนื้อหาของ layer อื่น

Common Pitfalls

- Server bound to localhost only
- Missing trunk
- Wrong VLAN
- OS firewall enabled
- Capture on wrong interface

Extension Activities

- Add TLS encryption

- Measure throughput vs latency
- Capture checksum errors
- Add checksum field to protocol
- Containerize server
- Compare with HTTP transfer